

Indice

Introduzione	2
1 Orientamento al Pensiero Computazionale	3
1.1 Cos'è il pensiero computazionale	3
1.2 Perché insegnarlo: obiettivi educativi e formativi	3
1.3 Metodologie didattiche per l'orientamento	3
1.3.1 Programmazione visuale e attività unplugged	3
1.3.2 Learning-by-doing e problem solving	3
1.3.3 Gamification e ambienti interattivi	4
1.3.4 Game-based learning	4
1.4 L'algoritmo Heap Sort come strumento didattico	4
1.4.1 Fondamenti teorici	4
1.4.2 Valore educativo dell'ordinamento a heap	5

Introduzione

L'informatica è una disciplina che, negli ultimi decenni, ha assunto un ruolo centrale e imprescindibile nella formazione degli studenti di ogni ordine e grado. Non si tratta più soltanto di apprendere l'utilizzo strumentale del calcolatore, ma di acquisire una nuova forma mentis, un modo di pensare strutturato e analitico volto alla risoluzione di problemi complessi: il **pensiero computazionale**. In questo scenario educativo in rapida evoluzione, l'apprendimento degli algoritmi fondamentali e delle strutture dati rappresenta un passaggio cruciale per comprendere le logiche profonde con cui le macchine elaborano, organizzano e trasformano le informazioni.

Tuttavia, lo studio di concetti astratti come gli algoritmi di ordinamento può risultare ostico per molti studenti se affrontato esclusivamente sui libri di testo. La mancanza di un riscontro visivo immediato e l'astrazione logica richiesta possono creare una barriera all'apprendimento. È qui che si inserisce il presente progetto, nato con l'obiettivo di colmare questo divario tra teoria e pratica attraverso l'uso delle moderne tecnologie web.

Questo progetto di tirocinio si propone di progettare e sviluppare un'applicazione web interattiva dedicata all'apprendimento dell'algoritmo **Heap Sort**. Sfruttando le potenzialità di HTML, CSS e JavaScript, e adottando un approccio basato sulla *gamification*, l'applicazione trasforma lo studio di una struttura dati complessa (lo Heap) in una sfida coinvolgente. L'utente non è più un osservatore passivo, ma diventa protagonista attivo del processo di ordinamento, sfidando un'intelligenza artificiale (CPU) in una gara di velocità e logica.

L'obiettivo didattico primario è rendere tangibili e manipolabili concetti che altrimenti rimarrebbero astratti, come la struttura ad albero binario, la verifica della proprietà di heap e le operazioni di scambio (swap) e riordinamento. Attraverso l'esperienza diretta, l'errore e la competizione ludica, lo studente può interiorizzare i meccanismi dell'algoritmo in modo naturale e duraturo.

Nel seguito di questo documento verranno dettagliati i principi teorici che hanno guidato la progettazione, l'analisi dei requisiti tecnici e didattici, le scelte architetturali (come l'adozione del pattern Model-View-Presenter) e le fasi di implementazione che hanno portato alla realizzazione del prodotto finale.

Capitolo 1

1 Orientamento al Pensiero Computazionale

1.1 Cos'è il pensiero computazionale

Il pensiero computazionale è un processo mentale che consente di formulare problemi e le loro soluzioni in modo che possano essere rappresentati come sequenze di istruzioni eseguibili da un agente che elabora informazioni (come un computer). Non è limitato alla programmazione, ma è una competenza trasversale che include:

- **Astrazione:** Semplificare la realtà per concentrarsi sugli aspetti essenziali.
- **Decomposizione:** Suddividere un problema complesso in parti più piccole e gestibili.
- **Riconoscimento di pattern:** Identificare somiglianze e regolarità nei dati.
- **Algoritmica:** Definire una serie di passi precisi per raggiungere un obiettivo.

1.2 Perché insegnarlo: obiettivi educativi e formativi

L'introduzione del pensiero computazionale nelle scuole risponde alla necessità di formare cittadini consapevoli in una società digitale. Gli obiettivi principali includono:

- Sviluppare la capacità di **analisi logica** e **problem solving**.
- Promuovere la **creatività** nel trovare soluzioni innovative.
- Incoraggiare la **perseveranza** e la capacità di gestire l'errore (debugging) come parte del processo di apprendimento.
- Fornire gli strumenti per passare da consumatori passivi a **creatori attivi** di tecnologia.

1.3 Metodologie didattiche per l'orientamento

Per insegnare efficacemente questi concetti, è necessario adottare metodologie attive che coinvolgano direttamente gli studenti.

1.3.1 Programmazione visuale e attività unplugged

La programmazione visuale (a blocchi) riduce il carico cognitivo legato alla sintassi del codice, permettendo di concentrarsi sulla logica. Le attività *unplugged* (senza computer) utilizzano giochi, carte o movimenti fisici per introdurre concetti informatici in modo analogico, rendendoli accessibili anche senza infrastrutture costose.

1.3.2 Learning-by-doing e problem solving

L'approccio "imparare facendo" è fondamentale. Invece di memorizzare definizioni, gli studenti affrontano problemi reali e costruiscono la loro conoscenza attraverso la sperimentazione pratica. L'errore diventa un'opportunità di apprendimento e non un fallimento.

1.3.3 Gamification e ambienti interattivi

La *gamification* applica meccaniche di gioco (punti, livelli, sfide) a contesti non ludici per aumentare la motivazione. Gli ambienti interattivi offrono feedback immediato, permettendo allo studente di vedere subito l'effetto delle proprie azioni, rinforzando il ciclo di apprendimento.

1.3.4 Game-based learning

A differenza della gamification, il *Game-Based Learning* utilizza veri e propri giochi come veicolo principale per l'apprendimento. Il gioco diventa il "libro di testo" interattivo, dove il contenuto didattico è intrinseco alle meccaniche di gioco stesse.

1.4 L'algoritmo Heap Sort come strumento didattico

1.4.1 Fondamenti teorici

L'Heap Sort è un algoritmo di ordinamento basato su confronto che utilizza una struttura dati chiamata **Heap** (o mucchio). Un Heap è un albero binario quasi completo che soddisfa la *proprietà di heap*.

Struttura e Proprietà Un albero binario si dice *quasi completo* se tutti i livelli, tranne eventualmente l'ultimo, sono completamente riempiti e tutti i nodi dell'ultimo livello sono il più a sinistra possibile. Questa proprietà permette di rappresentare efficientemente l'heap tramite un array A , dove per ogni nodo all'indice i :

- Il figlio sinistro si trova a $2i + 1$
- Il figlio destro si trova a $2i + 2$
- Il genitore si trova a $\lfloor \frac{i-1}{2} \rfloor$

Esistono due varianti principali di Heap:

- **Max-Heap:** Per ogni nodo i diverso dalla radice, il valore del nodo è minore o uguale a quello del padre: $A[\text{parent}(i)] \geq A[i]$. La radice contiene il valore massimo.
- **Min-Heap:** Per ogni nodo i diverso dalla radice, il valore del nodo è maggiore o uguale a quello del padre: $A[\text{parent}(i)] \leq A[i]$. La radice contiene il valore minimo.

Algorithm 1: Max-Heapify

Input: Array A , indice i

Output: Sottoalbero radicato in i conforme alla proprietà di Max-Heap

```
1  $left \leftarrow 2i + 1$ ;  
2  $right \leftarrow 2i + 2$ ;  
3  $largest \leftarrow i$ ;  
4 if  $left < A.heapSize \wedge A[left] > A[largest]$  then  
5   |  $largest \leftarrow left$ ;  
6 end  
7 if  $right < A.heapSize \wedge A[right] > A[largest]$  then  
8   |  $largest \leftarrow right$ ;  
9 end  
10 if  $largest \neq i$  then  
11   | Swap  $A[i]$  with  $A[largest]$ ;  
12   | Heapify( $A, largest$ );  
13 end
```

Procedura Heapify La procedura **Heapify** è fondamentale per mantenere la proprietà di heap. Essa assume che gli alberi binari radicati nei figli del nodo i siano già degli heap, ma che $A[i]$ possa essere più piccolo dei suoi figli, violando la proprietà di Max-Heap. La procedura fa "scendere" il valore di $A[i]$ nell'albero fino a ripristinare la proprietà corretta. La complessità temporale di **Heapify** è $O(\log n)$, proporzionale all'altezza dell'albero.

L'Algoritmo Heap Sort L'algoritmo **Heap Sort** combina la costruzione dell'heap e l'estrazione ripetuta della radice per ordinare l'array.

Algorithm 2: Heap Sort

```

Input: Array  $A$  da ordinare
Output: Array  $A$  ordinato
/* Costruzione dell'Heap (Build-Max-Heap)                                */
1 for  $i \leftarrow \lfloor A.length/2 \rfloor - 1$  to 0 do
2   |   Max-Heapify( $A, i$ );
3 end
/* Fase di ordinamento                                                */
4 for  $i \leftarrow A.length - 1$  to 1 do
5   |   Swap  $A[0]$  with  $A[i]$ ;
6   |    $A.heapSize \leftarrow A.heapSize - 1$ ;
7   |   Max-Heapify( $A, 0$ );
8 end

```

L'algoritmo si divide in due fasi principali:

1. **Build Heap:** Si costruisce un Max-Heap partendo dall'array disordinato. Si itera dai nodi interni verso la radice applicando **Heapify**.
2. **Estrazione e Ordinamento:** Poiché la radice ($A[0]$) di un Max-Heap contiene sempre il valore massimo, lo si scambia con l'ultimo elemento dell'array ($A[i]$), riducendo la dimensione dell'heap considerato di 1. Il nuovo elemento in $A[0]$ potrebbe violare la proprietà di heap, quindi si chiama **Heapify** sulla radice per ripristinarla. Si ripete il processo fino a quando l'heap è vuoto.

La complessità totale dell'Heap Sort è $O(n \log n)$, rendendolo un algoritmo di ordinamento molto efficiente, anche nel caso peggiore.

1.4.2 Valore educativo dell'ordinamento a heap

L'Heap Sort si presta eccellentemente alla didattica per diversi motivi:

- **Visualizzazione:** La struttura ad albero semplifica la comprensione delle relazioni gerarchiche tra i dati rispetto alla sola visualizzazione lineare di un array.
- **Complessità:** Introduce concetti di efficienza algoritmica ($O(n \log n)$) in modo intuitivo, mostrando come una struttura dati intelligente possa velocizzare le operazioni rispetto a metodi più semplici ma lenti (come il Bubble Sort).
- **Pensiero Ricorsivo:** La procedura di *heapify* (ripristino dell'ordine) è un classico esempio di logica ricorsiva o iterativa applicata a sotto-alberi.